PHP's part: it will decode the request body, build the corresponding array structure, and expose it via the `$_POST` superglobal. Here's some slightly more complex form-encoded data:

```
$ curl -X 'POST' -d 'animals[cat]=jess&humans[]=anna&humans[]=paul' \
  'http://localhost:8080/index.php'
...
array(2) {
  ["animals"]=>
  array(1) {
    ["cat"]=>
    string(4) "jess"
  }
  ["humans"]=>
  array(2) {
    [0]=>
    string(4) "anna"
    [1]=>
    string(4) "paul"
  }
}
```

As you can see, the syntax of `x-www-form-urlencoded` request data works just like the previous query string examples. This way of passing data in POST (and PUT) requests is baked into HTML as the `<form>` element, which will submit data via a POST request with minimal setup https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/form. As mentioned above, the automatic decoding of the request data and population of the `$_POST` superglobal depends on the `Content-Type:` HTTP header, which must be set to `application/x-www-form-urlencoded` or `multipart/form-data`. `x-www-form-urlencoded` is used for simple form-based data exchange (key/value pairs) as demonstrated here; `multipart/form-data` is for more complex scenarios including the transmission of binary data (such as files). We'll be looking at this in more detail very shortly.

Another very common approach for sending data to servers is to use JSON, especially when making calls that don't rely on user input (and therefore wouldn't have an HTML form in the loop). As a reminder, here's the code for `index.php`…